Relazione progetto Parallel Computing: Boids

Implementazione in C e Parallelizzazione OpenMP

Piergiorgio Danti 7196648
Università degli Studi di Firenze

14 aprile 2026

Indice

1 Abstract **2**

2 Introduzione **2**
2.1 Boids 2
2.2 Sfide 2

3 Descrizione dell'algoritmo **2**
3.1 Spiegazione dell'algoritmo sequenziale (ed ottimizzazioni proposte) 2
3.2 Identificazione delle sezioni parallelizzabili 4

4 Implementazione parallela **4**

5 Setup sperimentale **5**

6 Risultati Sperimentali e Analisi delle Performance **5**
6.1 Confronto tra le Implementazioni 5
6.2 Analisi di Scalabilità 6
6.3 Speedup ed Efficienza Parallela 6
6.4 Analisi del CPU Time e dell'Overhead 8
6.5 Comportamento al Variare del Numero di Boids 8
6.6 Analisi di Weak Scaling 8
6.7 Impatto dello Scheduling OpenMP 8

7 Ulteriori analisi e verifiche **10**
7.1 Impatto delle Ottimizzazioni del Compilatore 10
7.2 Validazione e Interfaccia Grafica 11

8 Discussioni **11**

1 Abstract

In questo elaborato è stato studiato un approccio alla parallelizzazione tramite l'utilizzo di OpenMP in C dell'algoritmo boids. In particolare, a partire da una versione sequenziale, sono state sperimentate diverse strategie di ottimizzazione e parallelizzazione, e sono stati riportati i relativi risultati. Nella versione migliore si riesce ad ottenere uno speedup di circa 5.5 rispetto alla versione sequenziale.

2 Introduzione

2.1 Boids

Boids è un algoritmo che permette di simulare il comportamento di uno stormo di uccelli in volo. Ogni individuo segue alcune regole che dipendono dalla posizione e dalla velocità dei suoi vicini. In particolare vengono seguite tre regole principali:

- **Separation**: ogni boids si allontana dai boids troppo vicini.
- **Alignment**: ogni boids modifica la propria velocità in base a quella dei vicini.
- **Cohesion**: ogni boids si muove verso il centro di massa dei suoi vicini.

Ci sono anche altre regole da rispettare, tra cui un limite massimo e minimo di velocità che, se violati, vengono corretti adeguatamente. Inoltre i boids non possono superare i bordi dello spazio di simulazione; se questo accade, la loro rotta viene modificata per farli tornare nella zona ammissibile.

2.2 Sfide

Si richiede che ogni soggetto aggiorni la propria posizione in funzione degli altri. L'obiettivo principale è quello di presentare una versione in cui ogni boids non debba accedere alle informazioni di tutti gli altri, ma solo a quelle dei vicini, e in cui questa operazione possa essere eseguita in parallelo. Queste modifiche richiedono la gestione di strutture dati di supporto e l'accesso in mutua esclusione a tali strutture nel modo più efficiente possibile.

3 Descrizione dell'algoritmo

3.1 Spiegazione dell'algoritmo sequenziale (ed ottimizzazioni proposte)

In primo luogo è stata implementata una versione dell'algoritmo sequenziale, senza ottimizzazioni. Per

Algorithm 1 Boids

Require: N numero di boids, T numero totale di frame
  Inizializza posizioni $\mathbf{P}$ e velocità $\mathbf{V}$ per ogni boids $b \in \{1, \ldots, N\}$
  while $frame < T$ **do**
    for all boid b_i in $\{1, \ldots, N\}$ **do**
      $vicini \leftarrow$ identifica boids nel raggio di visione di b_i
      Separation($b_i, vicini$)
      Alignment($b_i, vicini$)
      Cohesion($b_i, vicini$)
      GestioneBordi o limite velocità
    end for
    $frame \leftarrow frame + 1$
  end while

fare in modo che ogni boids aggiorni la propria posizione rispetto a quella degli altri in maniera "contemporanea", tutti devono fare riferimento alla versione dell'aggiornamento precedente. In pratica, tutti leggono da una versione "old" della lista dei boids e, al termine dell'iterazione, viene prodotta una versione "new" che contiene le posizioni e le velocità aggiornate.

Questa versione dell'algoritmo (1) presentava però il problema di avere una complessità quadratica rispetto al numero di boids, in quanto ognuno di essi doveva controllare la posizione di tutti gli altri per capire se fossero nel proprio raggio di interesse, rendendo il calcolo molto oneroso. Per questo motivo è stata proposta una versione con l'utilizzo dello spatial partitioning, nella quale si limita il numero di controlli effettuati da ogni soggetto. Dividendo lo spazio in celle, ognuna grande quanto il visual range (cioè il raggio entro il quale si applicano Alignment e Cohesion), per ogni boids è sufficiente controllare la posizione di quelli che si trovano nella sua cella e nelle 8 celle adiacenti. In questo modo la complessità si riduce notevolmente.

Per applicare questo miglioramento sono state utilizzate alcune strutture dati aggiuntive che permettono di gestire le celle. Infatti, ad ogni iterazione, è necessario calcolare in quale cella si trova ogni boid. Per poter visualizzare facilmente tutti i boids di una cella, i loro indici vengono salvati in modo parzialmente ordinato in un array di indici. Di conseguenza, prima di iterare su ogni boids per aggiornare la posizione, è necessario aggiornare le strutture dati delle celle (come illustrato in 2).

Algorithm 2 Spatial Partitioning e Grid Sorting

Require: $boids, num_boids, cols, rows$

// **Fase 1: Conteggio boids per ogni cella**
for $i = 0$ **to** $num_boids - 1$ **do**
  $idx \leftarrow$ get_cell_index$(boids[i].x, boids[i].y, cols, rows)$
  $counting_cell[idx] \leftarrow counting_cell[idx] + 1$
end for
// **Fase 2: Calcolo degli Offset**
$cell_offsets[0] \leftarrow 0$
for $i = 0$ **to** $total_cells - 1$ **do**
  $cell_offsets[i + 1] \leftarrow cell_offsets[i] + counting_cell[i]$
end for
// **Fase 3: Ordinamento degli indici dei boids**
$temp_ptr \leftarrow$ copia di $cell_offsets$
for $i = 0$ **to** $num_boids - 1$ **do**
  $idx \leftarrow$ get_cell_index$(boids[i].x, boids[i].y, cols, rows)$
  $pos \leftarrow temp_ptr[idx]$
  $sorted_ind[pos] \leftarrow i$
  $temp_ptr[idx] \leftarrow temp_ptr[idx] + 1$
end for

Come mostrato in 3, per poter sfruttare il principio di località spaziale si ordina anche la lista dei boids in memoria. D'altro canto, ordinare decine di migliaia di elementi in memoria ad ogni iterazione può introdurre un overhead non indifferente. Per questo motivo l'ordinamento viene effettuato solo con una certa frequenza (per esempio ogni 10 iterazioni), sfruttando il fatto che, dopo poche iterazioni, è molto probabile che ogni boids rimanga nella stessa cella in cui si trova, o al massimo in una di quelle adiacenti.

Algorithm 3 Boids Sorting

Require: $boids$ (array originale), $sorted_ind$ (mappa degli indici)
Ensure: $sorted_boids$

```
for i = 0 to num_boids − 1 do
    idx ← sorted_ind[i]
    sorted_boids[i].pos ← boids[idx].pos
    sorted_boids[i].vel ← boids[idx].vel
end for
return sorted_boids
```

3.2 Identificazione delle sezioni parallelizzabili

In 1 il ciclo `for` è adatto alla parallelizzazione, poiché ogni thread può calcolare la nuova posizione dei boids di propria competenza in modo indipendente e contemporaneo. In 2 sono state parallelizzate la fase 1 e la fase 3, mentre la fase 2 è stata mantenuta sequenziale, poiché presenta una dipendenza dall'indice $i - 1$. Inoltre, questa fase opera sul numero di celle che è sensibilmente inferiore rispetto al numero di boids, rendendo poco conveniente una sua parallelizzazione. Anche l'aggiornamento delle posizioni in memoria dei boids in 3 è stato parallelizzato. Altre operazioni, come l'inizializzazione di posizioni e velocità, pur parallelizzabili, non influendo sul tempo di calcolo, sono rimaste sequenziali.

4 Implementazione parallela

Per la parallelizzazione del ciclo in 1 è stata utilizzata la direttiva `#pragma omp parallel for schedule (runtime)`, per poter applicare diverse strategie di scheduling senza dover ricompilare. Tuttavia, ci si aspetta che una strategia `dynamic` possa portare risultati migliori, poiché la posizione e la concentrazione dei boids nello spazio non è uniforme e questo scheduling consente una gestione più flessibile del carico di calcolo.

La gestione parallela di 2 ha permesso di implementare diverse strategie. In particolare, per le fasi 1 e 3 sono stati testati due approcci per la gestione della sezione critica nelle risorse condivise:

1. Utilizzo di istogrammi locali.
2. Utilizzo della direttiva `#pragma atomic`.

Nella prima versione, ogni thread utilizza un istogramma locale per contare quanti boids appartengono a ciascuna cella della griglia. Gli istogrammi sono allocati con padding e opportunamente allineati in memoria per evitare false sharing, impedendo che istogrammi appartenenti a thread diversi condividano la stessa linea di cache. Il conteggio avviene tramite due cicli parallelizzati. Nel primo ciclo i thread si dividono l'insieme dei boids e aggiornano il proprio istogramma locale incrementando il contatore della cella corrispondente a ciascun boid. Nel secondo ciclo viene effettuata una riduzione sulle celle: per ogni cella si sommano i contributi di tutti gli istogrammi locali, ottenendo il numero totale di boids presenti in quella cella, memorizzato nell'array `counting_cell`. Successivamente vengono calcolati gli offset delle celle, all'interno di una clausola `single`. A questo punto ogni thread costruisce un proprio array di offset privati. L'offset di una cella per un determinato thread è dato dall'offset globale della cella più il numero di boids della stessa cella contati dai thread con identificatore minore. In questo modo ogni thread ottiene una porzione distinta dell'intervallo associato alla cella all'interno di `sorted_ind`. Infine gli indici dei boids vengono inseriti nell'array `sorted_ind`. Ogni thread utilizza i propri offset privati per scrivere nella porzione di memoria assegnata, incrementando localmente l'offset dopo ogni inserimento. Questo permette di evitare sincronizzazioni esplicite tra i thread.

La seconda versione utilizza la gestione della risorsa condivisa tramite la clausola `atomic`. In particolare, nelle fasi 1 e 3, eseguite in parallelo, gli aggiornamenti avvengono con operazioni `atomic`, mentre nella terza fase si utilizza `atomic capture`, per eseguire in maniera atmoica le operazioni di lettura dell'offset e il suo incremento.

Per quanto riguarda la memorizzazione dei boids in memoria, sono state testate entrambe le strategie Structure of Array (SoA) e Array of Structures (AoS). Per consentire l'utilizzo di entrambe le rappresentazioni all'interno dello stesso codice sono state definite delle macro di accesso ai campi dei boids. In base alla configurazione di compilazione tali macro accedono ai dati secondo lo specifico layout. SoA è particolarmente adatta alle parti di codice in cui si calcolano esclusivamente le posizioni, evitando il caricamento dei campi relativi alle velocità. In entrambe le versioni i dati sono stati salvati sfruttando padding, per garantire allineamento e ridurre cachemiss.

5 Setup sperimentale

Le simulazioni per la raccolta delle metriche di performance sono state eseguite su un sistema dotato di processore Intel(R) Core(TM) i7-1165G7 (11th Gen). La CPU dispone di 4 core fisici e 8 thread logici grazie alla tecnologia Hyper-Threading. Il sistema possiede una gerarchia di cache composta da 320 KiB di cache L1, 5 MiB di cache L2 e 12 MiB di cache L3 condivisa. La memoria principale è di 16 GiB DDR4 a 3200 MHz. Il sistema operativo utilizzato è Linux x86_64 con kernel 6.17.0.

Per valutare le prestazioni sono state testate diverse versioni dell'implementazione. In particolare sono state considerate due versioni sequenziali (AoS e SoA) e quattro versioni parallele, ottenute combinando le due organizzazioni dei dati con le due strategie utilizzate per gestire l'accesso concorrente alle strutture condivise.

Le simulazioni sono state eseguite su configurazioni con diversi numeri di boids e thread. Durante le esecuzioni sono stati misurati sia il wall time sia il CPU time, considerando esclusivamente le parti di calcolo dell'algoritmo.

Per facilitare il test delle diverse versioni, il codice è stato mantenuto negli stessi file ma organizzato in modo da separare logicamente le varie implementazioni. La versione da utilizzare viene selezionata tramite direttive di compilazione condizionale del preprocessore C, evitando la necessità di modificare manualmente il codice o duplicare i file.

La gestione della compilazione è stata realizzata tramite CMake. Il file di configurazione definisce diversi target eseguibili che corrispondono alle varie versioni dell'implementazione (sequenziali e parallele). Tutti i target condividono gli stessi file sorgente principali (`main.c`, `boids_simulation.c`, `boids_core.c` e `boids_manager.c`). Le versioni parallele abilitano OpenMP tramite il flag `-fopenmp`. Tutti i target sono compilati con le stesse opzioni di ottimizzazione (`-O3`, `-march=native`, `-ffast-math` e `-flto`), escluso un target, compilato senza ottimizzazioni.

6 Risultati Sperimentali e Analisi delle Performance

L'implementazione descritta nelle sezioni precedenti è stata valutata attraverso una serie di test sperimentali finalizzati ad analizzare il comportamento delle diverse varianti dell'algoritmo. In particolare, sono state confrontate diverse scelte implementative per determinare la configurazione più efficiente.

6.1 Confronto tra le Implementazioni

Sono state confrontate le quattro varianti dell'algoritmo parallelo. Come mostrato in Figura 1, la con-

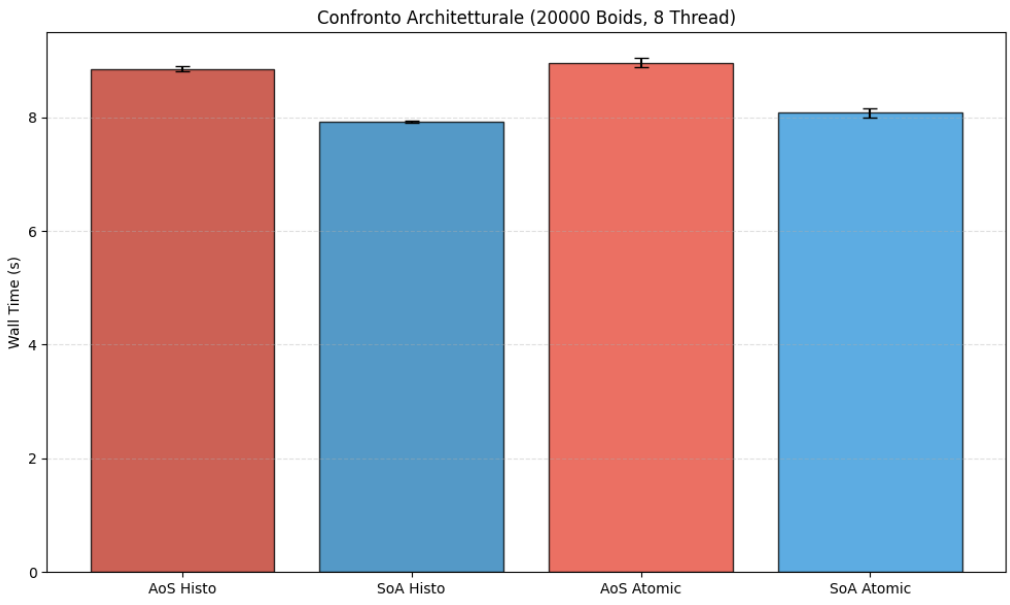

Figura 1: Confronto del Wall Time medio (20.000 boids, 8 thread) tra le diverse implementazioni.

figurazione che utilizza il layout SoA insieme alla strategia basata su istogrammi risulta essere la più efficiente.

L'organizzazione dei dati in formato SoA permette infatti di migliorare l'utilizzo della cache: durante il calcolo delle interazioni tra boids vengono frequentemente utilizzate solo le coordinate spaziali, e avere tali dati memorizzati in vettori contigui riduce il numero di accessi non necessari alla memoria.

Per quanto riguarda la gestione delle strutture condivise, l'approccio basato su istogrammi locali consente di ridurre la contesa tra thread rispetto all'utilizzo di operazioni `atomic`. Ogni thread lavora inizialmente su strutture dati private e solo in una fase successiva i risultati vengono combinati, riducendo il numero di sincronizzazioni necessarie.

6.2 Analisi di Scalabilità

Per valutare la scalabilità dell'implementazione parallela è stato eseguito un test di strong scaling, mantenendo fisso il numero di boids (20.000) e variando il numero di thread utilizzati. Come si può osservare dalla Figura 2, aumentando il numero di thread si ottiene una riduzione significativa del tempo di esecuzione fino a 8 thread. Questo risultato è coerente con le caratteristiche hardware del sistema utilizzato per i test, che dispone di 4 core fisici e 8 thread logici.

Quando il numero di thread viene portato a 16 si osserva invece un peggioramento delle prestazioni. In questo caso l'aumento dei thread non comporta un effettivo incremento delle risorse computazionali disponibili, ma introduce principalmente overhead dovuto alla gestione dei thread e alla contesa per le unità di esecuzione dei core.

6.3 Speedup ed Efficienza Parallela

Per analizzare più nel dettaglio il comportamento della versione parallela sono stati calcolati lo speedup e l'efficienza rispetto alla versione sequenziale. Dai risultati riportati in Figura 3 si osserva che lo speedup cresce in modo quasi lineare fino a 4 thread. Con 8 thread si ottiene comunque un miglioramento significativo, anche se l'efficienza tende a diminuire a causa dei costi di overhead.

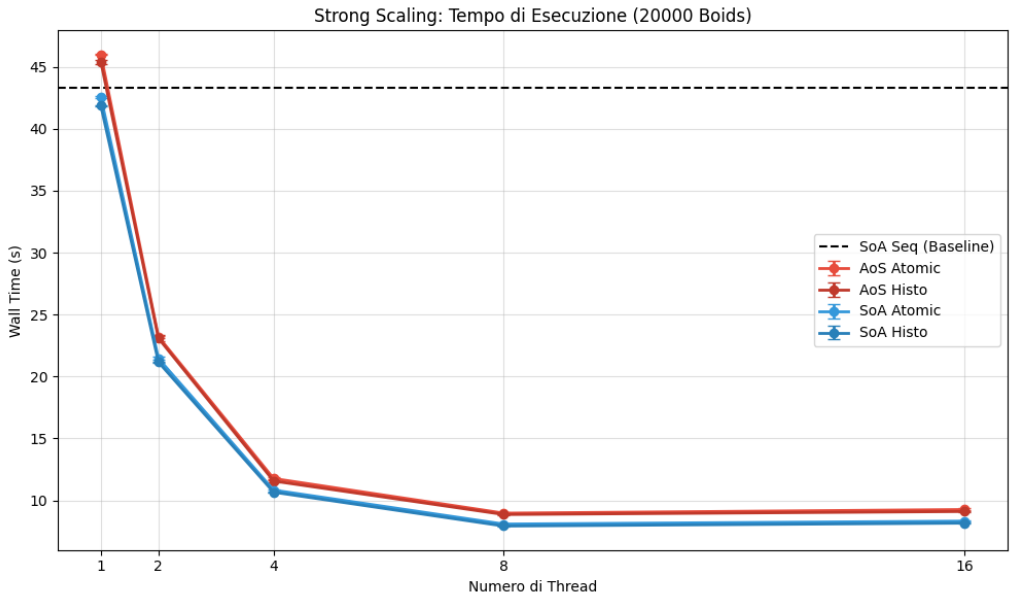

Figura 2: Analisi di Strong Scaling: Wall Time al variare del numero di thread.

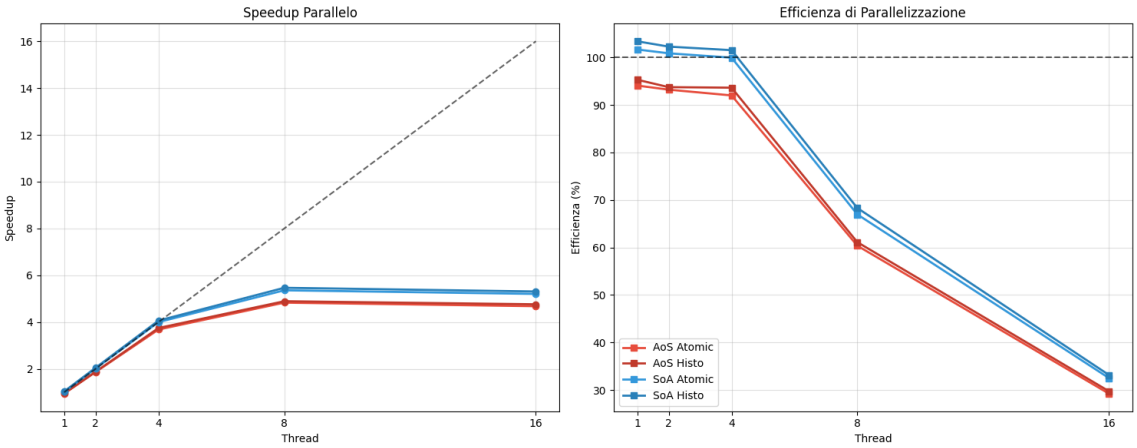

Figura 3: Analisi dello Speedup (sinistra) e dell'Efficienza (destra) per 20.000 boids.

6.4 Analisi del CPU Time e dell'Overhead

Per capire meglio quanto pesa l'overhead introdotto dalla parallelizzazione, è stato confrontato il CPU time totale misurato con il CPU time ideale, cioè quello che ci si aspetterebbe in condizioni teoriche senza costi aggiuntivi. Come mostrato in Figura 4, fino a 4 thread i due valori sono molto simili. Oltre

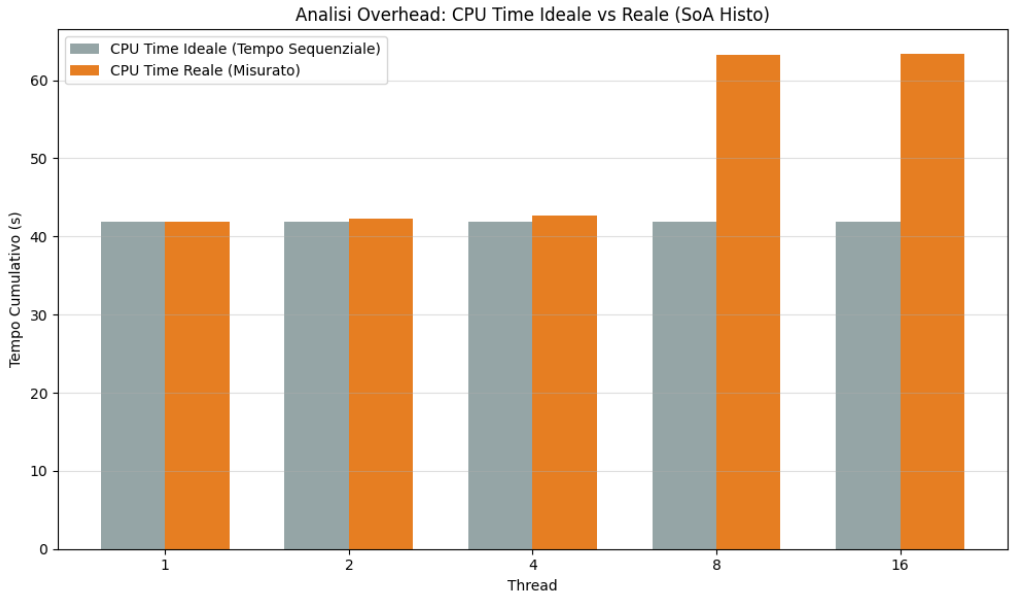

Figura 4: Confronto tra CPU Time Ideale e CPU Time Reale misurato (versione SoA Histo).

gli 8 thread, il CPU time reale cresce rispetto a quello ideale, perché una parte sempre maggiore del tempo viene usata per la gestione del parallelismo invece che per il calcolo.

6.5 Comportamento al Variare del Numero di Boids

Un ulteriore test è stato effettuato variando il numero di boids presenti nella simulazione, mantenendo fisso il numero di thread pari a 8. Come mostrato in Figura 5, il tempo di esecuzione cresce in modo approssimativamente lineare all'aumentare del numero di boids. Questo comportamento è dovuto all'utilizzo della tecnica di spatial partitioning, che consente di limitare il numero di confronti tra individui evitando la complessità quadratica che caratterizzerebbe un approccio basato sul confronto tra tutte le coppie di boids.

6.6 Analisi di Weak Scaling

Per valutare il comportamento dell'algoritmo all'aumentare simultaneo del numero di thread e del carico computazionale è stato eseguito un test di weak scaling. In questo esperimento il numero di boids per thread viene mantenuto approssimativamente costante, aumentando proporzionalmente il numero totale di boids. Come mostrato in Figura 6, il tempo di esecuzione rimane relativamente stabile passando da 1 a 2 thread, mentre aumentando ulteriormente il numero di thread si osserva un progressivo incremento del tempo di esecuzione, che si discosta dall'andamento ideale costante che si avrebbe in condizioni di weak scaling perfetto.

6.7 Impatto dello Scheduling OpenMP

Infine è stato analizzato l'impatto delle diverse strategie di scheduling offerte da OpenMP sul tempo di esecuzione dell'algoritmo. A causa della distribuzione non uniforme dei boids nello spazio, il carico computazionale associato a ciascun individuo può variare significativamente durante la simulazione. In

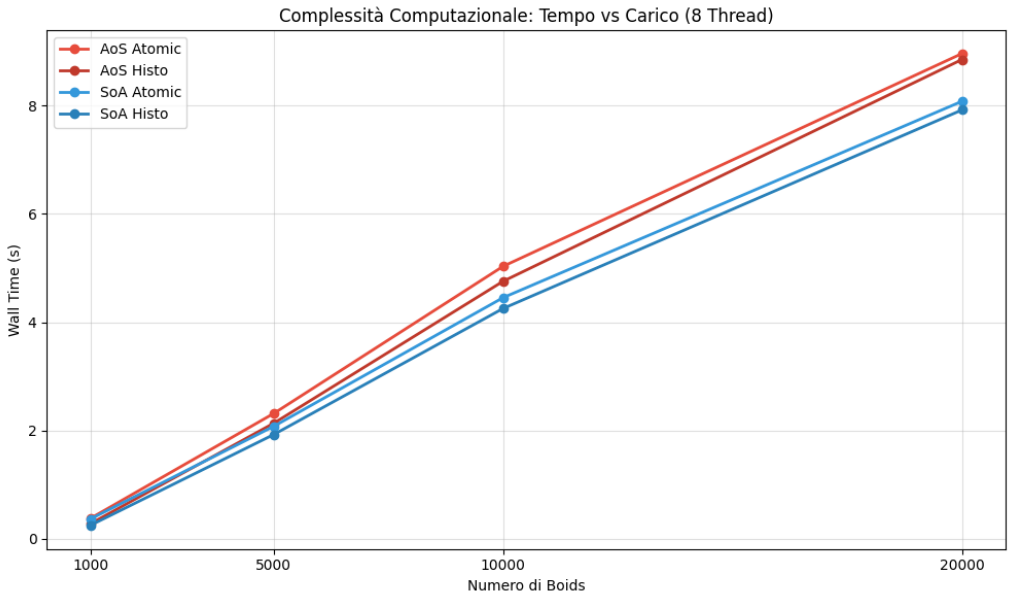

Figura 5: Wall Time al variare del numero di boids con 8 thread.

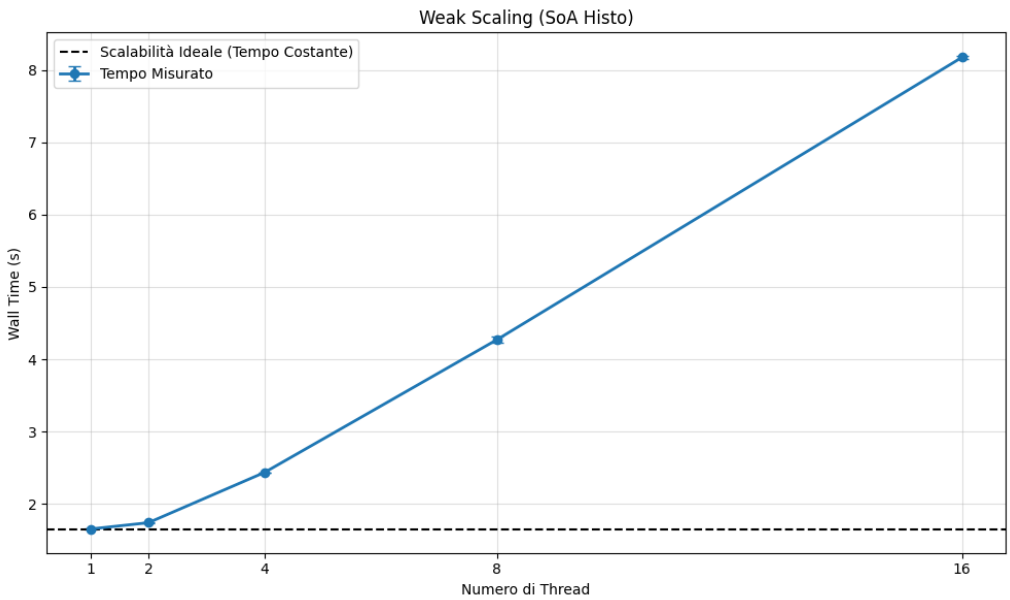

Figura 6: Analisi di Weak Scaling (SoA con istogrammi). Rapporto boids su threads di 1250.

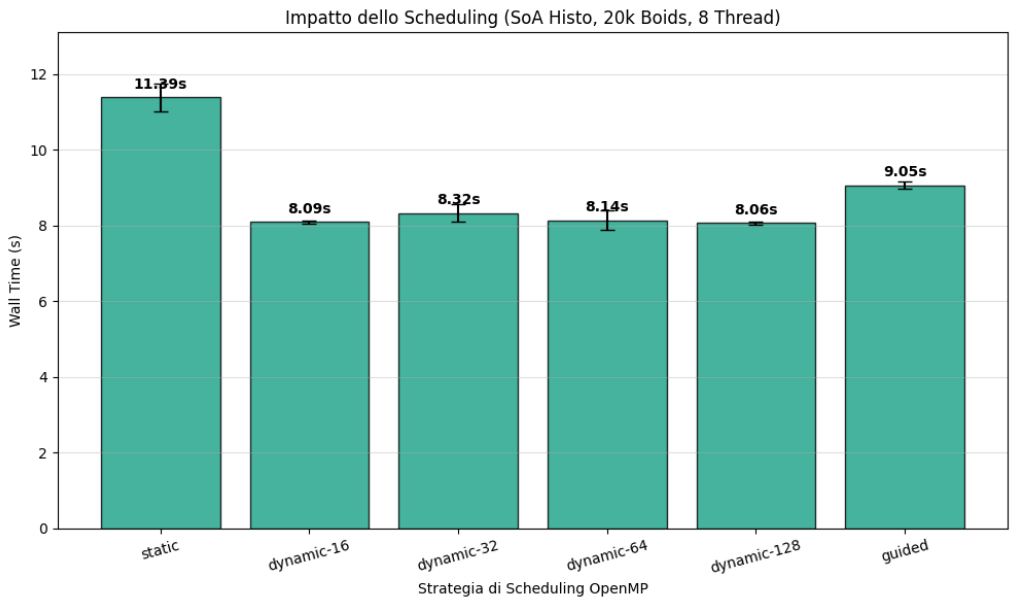

Figura 7: Confronto tra diverse strategie di scheduling (SoA con istogrammi, 20.000 boids, 8 thread).

queste condizioni, uno scheduling di tipo `static` può portare a situazioni di sbilanciamento del carico tra i thread.

Come si osserva in Figura 7, le strategie `dynamic` permettono di ottenere prestazioni migliori, poiché consentono una distribuzione più flessibile delle iterazioni tra i thread.

7 Ulteriori analisi e verifiche

7.1 Impatto delle Ottimizzazioni del Compilatore

Oltre alle scelte implementative, è stato valutato anche l'impatto delle ottimizzazioni del compilatore sulle prestazioni. Come mostrato in Figura 8, l'utilizzo di flag di ottimizzazione comporta un

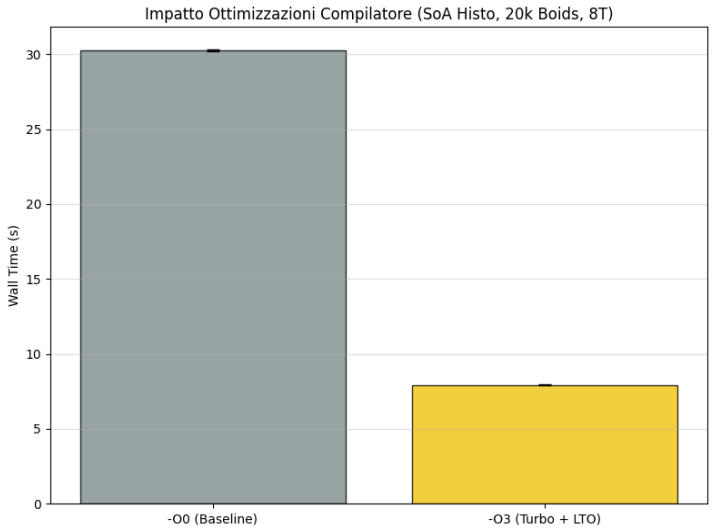

Figura 8: Confronto tra compilazione non ottimizzata (-O0) e compilazione ottimizzata (-O3).

miglioramento significativo delle prestazioni rispetto alla versione compilata senza.

7.2 Validazione e Interfaccia Grafica

Per verificare visivamente il corretto funzionamento dell'algoritmo e osservare il comportamento dello stormo, è stata realizzata anche una versione con interfaccia grafica utilizzando la libreria raylib. Questa componente consente di visualizzare in tempo reale l'evoluzione della simulazione e il movimento dei boids nello spazio.

È importante sottolineare che il costo computazionale legato alla grafica non è stato considerato nelle misure di performance riportate nelle sezioni precedenti. I benchmark sono stati infatti eseguiti utilizzando esclusivamente la versione senza rendering, in modo da misurare solamente il tempo necessario al calcolo della simulazione.

Per verificare la correttezza delle implementazioni parallele è stata inoltre sviluppata una semplice procedura di validazione. Tale procedura confronta le posizioni e le velocità finali dei boids ottenute dalle diverse versioni dell'algoritmo con quelle prodotte dalla versione sequenziale di riferimento. I risultati del confronto sono riportati nel Listing 1.

```
boids_seq_soa           | PASS (Max Diff: 0.00e+00)
boids_par_aos_histo     | PASS (Max Diff: 0.00e+00)
boids_par_soa_histo     | PASS (Max Diff: 0.00e+00)
boids_par_aos_atomic    | PASS (Max Diff: 6.10e-05)
boids_par_soa_atomic    | PASS (Max Diff: 1.60e-04)
```

Listing 1: Risultati della validazione (1000 boids, 10 iterazioni, 8 thread, tolleranza 10^{-2}).

Dai risultati si osserva che le versioni parallele basate su istogrammi producono risultati identici a quelli della versione sequenziale entro la precisione numerica considerata.

Le versioni che utilizzano operazioni `atomic`, invece, mostrano leggeri differenze. Queste sono dovute principalmente al fatto che l'ordine con cui i thread eseguono alcune operazioni di aggiornamento può variare tra un'esecuzione e l'altra. Poiché l'aritmetica in virgola mobile non è perfettamente associativa, variazioni nell'ordine delle operazioni possono produrre risultati numericamente leggermente diversi.

Nel contesto della simulazione dei boids, tali differenze risultano comunque trascurabili e non alterano il comportamento globale dello stormo osservato durante la simulazione.

8 Discussioni

I risultati ottenuti confermano che la parallelizzazione tramite OpenMP migliora significativamente le prestazioni rispetto alla versione sequenziale.

Dai test emerge quanto segue: l'uso di istogrammi locali riduce la contesa tra thread rispetto alle operazioni `atomic`, e la struttura SoA migliora l'utilizzo della cache. La combinazione di SoA con istogrammi locali rappresenta la configurazione migliore, offrendo il massimo speedup ed efficienza.

Dai test si vede che lo speedup massimo è circa 5.5, quindi più alto dei core fisici disponibili. Questo succede perché l'algoritmo fa molti accessi ai dati in memoria e i core spesso devono aspettare che i dati arrivino. In questo scenario l'Hyper-Threading aiuta molto, infatti quando un thread è fermo in attesa della memoria, il processore può eseguire le istruzioni dell'altro thread sullo stesso core. In questo modo il tempo di attesa viene sfruttato meglio e la CPU rimane più attiva, permettendo di ottenere prestazioni anche oltre quelle dei soli 4 core fisici.

Molti dei fattori intrinseci dell'algoritmo dipendono da parametri di configurazione, come il visual

range e il numero di celle, dimensionati appositamente in funzione del numero di boids per ottenere prestazioni scalabili. Ad esempio, la frequenza di riordino in memoria è stata fissata sperimentalmente a 10 iterazioni. Incrementando ulteriormente il numero di boids, potrebbe rendersi necessario limitare la quantità totale di celle: una griglia troppo densa, infatti, rischierebbe di introdurre un overhead eccessivo nella gestione delle strutture dati. Di conseguenza, per sviluppi futuri, potrebbe essere interessante approfondire i test sperimentando diverse configurazioni di questi parametri per analizzare che comportamento si può ottenere e capire se è possibile migliorare ulteriormente lo speedup.

Riferimenti bibliografici

[1] V. Hunter Adams, *Boids Algorithm*, Cornell University. Disponibile all'indirizzo: https://vanhunteradams.com/Pico/Animal_Movement/Boids-algorithm.html

[2] Ray Sanjuan et al., *raylib: A simple and easy-to-use library to enjoy videogames programming*. Disponibile all'indirizzo: https://www.raylib.com/

[3] OpenMP Architecture Review Board, *The OpenMP API specification for parallel programming*. Disponibile all'indirizzo: https://www.openmp.org/